POSTGRESQL / JSONB

ОТЧЕТ К РАБОТЕ № 1

Создание и изменение JSONB-документов

Бронирования авиаперевозок

Реляционные данные, JSONB и часовые пояса

2026

1. Цель и постановка задачи

Цель работы - освоить создание, чтение и изменение JSONB-документов средствами SQL в PostgreSQL на основе реляционных данных базы «Авиаперевозки».

Общее задание

Создать таблицу bookings_json с первичным ключом id, внешним ключом booking_id, полем data типа JSONB и временем создания created_at. Для каждого бронирования сформировать документ с обязательными полями booking_number, status и массивом passengers с полями id и name. Номер брони и сведения о пассажирах получить из исходной базы.

Индивидуальное задание

Для броней, содержащих рейсы в/из аэропортов, часовой пояс которых отличается от московского более чем на 6 часов, добавить комментарий в notes. Проверяются оба аэропорта каждого рейса. Достаточно одного подходящего рейса в бронировании; разница ровно 6 часов условию не соответствует.

Средства и исходные данные

Использованы PostgreSQL 18, SQL, утилита psql для загрузки и выполнения скриптов и pgAdmin 4 для просмотра результатов. Источник - официальная демобаза demo-small от 15.08.2017 [1]. База загружена из SQL-дампа; исходные таблицы не изменялись. В копии дампа исключены команда удаления базы и устаревшая настройка default_with_oids.

Исходная таблица	Назначение
bookings	Номер бронирования book_ref
tickets	Связь с бронью, passenger_id и passenger_name
ticket_flights	Связь билета ticket_no с рейсом flight_id
flights	Аэропорты и плановое время вылета и прилета
airports_data	Код аэропорта и его часовой пояс timezone

Все исходные таблицы находятся в схеме bookings базы demo. Каждая строка новой таблицы соответствует одному бронированию.

2. Создание и заполнение JSON-документов

Первый этап реализован файлом 01_create_and_fill.sql (полный текст приведен в приложении А).

Поле новой таблицы	Тип и ограничения
id	bigint; автоматически формируемый первичный ключ
booking_id	char(6); NOT NULL, UNIQUE; ссылка на bookings.book_ref
data	jsonb; NOT NULL; объект с обязательными ключами
created_at	timestamptz; по умолчанию CURRENT_TIMESTAMP

Ограничения CHECK проверяют, что data является объектом, содержит booking_number, status и passengers, а passengers имеет тип массива. Уникальность booking_id обеспечивает один документ на бронь.

Сначала SELECT DISTINCT получает уникальные сочетания номера брони, идентификатора и имени пассажира из tickets. Затем jsonb_build_object создает объект пассажира, а jsonb_agg собирает такие объекты в массив для каждой группы book_ref. Порядок элементов задается ORDER BY.

```
jsonb_agg(
    jsonb_build_object('id', passenger_id,
                       'name', passenger_name)
    ORDER BY passenger_id, passenger_name
) AS passengers
```

Бронирования соединяются с массивами пассажиров через LEFT JOIN. Функция jsonb_build_object формирует итоговый документ; COALESCE подставляет пустой массив при отсутствии пассажиров. Номер брони сохраняется и во внешнем ключе, и внутри JSON.

```
jsonb_build_object(
    'booking_number', b.book_ref,
    'status', 'confirmed',
    'passengers', COALESCE(p.passengers, '[]'::jsonb),
    'notes', ''
)
```

Статус confirmed задан условно, поскольку исходная таблица бронирований не содержит статуса; это допускается заданием. notes первоначально пуст. ON CONFLICT (booking_id) DO NOTHING сохраняет существующие документы при повторном запуске. Получено 262 788 JSON-документов.

3. Выполнение индивидуального задания

Обновление реализовано файлом 02_update_notes.sql (приложение Б). Через JOIN строится цепочка tickets → ticket_flights → flights → airports_data. Таблица аэропортов присоединяется дважды: для вылета (dep) и прилета (arr). SELECT DISTINCT оставляет уникальные номера броней.

Для вылета вычисляется разница местного времени аэропорта и московского в один и тот же момент scheduled_departure. Аналогичная проверка прилета использует scheduled_arrival. Это учитывает смещение пояса на дату рейса.

```
abs(extract(epoch FROM (
    (f.scheduled_departure AT TIME ZONE dep.timezone) -
    (f.scheduled_departure AT TIME ZONE 'Europe/Moscow')
)) / 3600) > 6
```

EXTRACT(EPOCH FROM ...) переводит интервал в секунды, деление на 3600 дает часы, ABS учитывает опережение и отставание. Проверки вылета и прилета соединены через OR. Аэропорты не сравниваются друг с другом: каждый из них сравнивается с Москвой.

UPDATE находит соответствующий документ по booking_id = book_ref. Функция jsonb_set изменяет только ключ notes. Старый текст сохраняется и объединяется с новым комментарием через перевод строки. Проверка POSITION предотвращает повторное добавление той же фразы.

Аэропорт	**Город**	**Разница, ч**
KHV	Хабаровск	7
KXK	Комсомольск-на-Амуре	7
VVO	Владивосток	7
GDX	Магадан	8
UUS	Южно-Сахалинск	8
DYR	Анадырь	9
PKC	Петропавловск-Камчатский	9

В реальных рейсах демобазы условию соответствуют перечисленные аэропорты. Комментарий добавлен к 18 799 бронированиям. Например, бронь 00000F содержит рейсы Москва - Южно-Сахалинск и обратно; разница относительно Москвы составляет 8 часов.

4. Запросы для демонстрации результата

После выполнения двух SQL-файлов итоговая таблица просматривается тремя самостоятельными запросами.

Общее количество JSON-документов

```
SELECT count(*) AS total_bookings
FROM bookings.bookings_json;
```

Результат: 262 788. COUNT(*) подсчитывает строки таблицы; каждая строка содержит один JSON-документ.

Количество документов с непустой заметкой

```
SELECT count(*) AS bookings_with_notes
FROM bookings.bookings_json
WHERE COALESCE(data ->> 'notes', '') <> '';
```

Результат: 18 799. Оператор ->> извлекает текст, COALESCE заменяет отсутствующее значение пустой строкой, а <> исключает пустые заметки. В данном наборе непустые notes содержат добавленный комментарий.

Просмотр первых 20 документов

```
SELECT booking_id,
       data ->> 'booking_number' AS booking_number,
       data ->> 'status' AS status,
       data -> 'passengers' AS passengers,
       data ->> 'notes' AS notes
FROM bookings.bookings_json
ORDER BY booking_id
LIMIT 20;
```

Поля JSON выводятся отдельными столбцами. Оператор -> сохраняет массив пассажиров в формате JSONB. ORDER BY задает сортировку по номеру брони, LIMIT ограничивает только вывод, не изменяя таблицу.

Пример структуры: бронь 00000F

Поле	Значение
booking_number	00000F
status	confirmed
passengers[0].id	1708 262537
passengers[0].name	PASSENGER_001

Имя пассажира в иллюстрации заменено условным обозначением.
notes: «Бронирование содержит рейс в/из аэропорта с разницей во времени относительно Москвы более 6 часов».

5. Вывод

В ходе лабораторной работы создана таблица bookings_json и заполнена на основе исходных бронирований и билетов. Освоено построение JSONB с помощью jsonb_build_object и jsonb_agg, извлечение значений операторами -> и ->>, а также изменение отдельного ключа через jsonb_set.

Сформировано 262 788 JSON-документов. Индивидуальное задание выполнено: для 18 799 броней, включающих рейсы в/из аэропортов с разницей относительно московского времени строго более 6 часов, добавлен комментарий в notes. Три запроса демонстрируют общее количество документов, количество документов с непустой заметкой и содержимое первых 20 записей. Повторное выполнение основных файлов не создает дубликатов документов и комментариев.

Список использованных источников

1. Postgres Professional. Демонстрационная база данных «Авиаперевозки», версия от 15.08.2017. https://postgrespro.ru/education/demodb

2. PostgreSQL. Документация: JSON Functions and Operators. https://www.postgresql.org/docs/18/functions-json.html

3. PostgreSQL. Документация: Date/Time Functions and Operators. https://www.postgresql.org/docs/18/functions-datetime.html

Состав приложений

Приложение А - полный текст 01_create_and_fill.sql.
Приложение Б - полный текст 02_update_notes.sql.
Три запроса для просмотра результата приведены в разделе 4.
Длинные строки листингов перенесены только для размещения на странице.

Приложение А

01_create_and_fill.sql

```
-- Этап 1: таблица и JSON-документ для каждого бронирования.
BEGIN;

CREATE TABLE IF NOT EXISTS bookings.bookings_json (
    id         bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    booking_id char(6) NOT NULL UNIQUE
               REFERENCES bookings.bookings(book_ref),
    data       jsonb NOT NULL,
    created_at timestamptz NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CHECK (jsonb_typeof(data) = 'object'),
    CHECK (data ?& ARRAY['booking_number', 'status', 'passengers']),
    CHECK (jsonb_typeof(data -> 'passengers') = 'array')
);

-- Один пассажир может иметь несколько билетов: сначала убираем повторы.
WITH unique_passengers AS (
    SELECT DISTINCT book_ref, passenger_id, passenger_name
    FROM bookings.tickets
), passenger_lists AS (
    SELECT book_ref,
           jsonb_agg(
               jsonb_build_object('id', passenger_id, 'name', passenger_name)
               ORDER BY passenger_id, passenger_name
           ) AS passengers
    FROM unique_passengers
    GROUP BY book_ref
)
INSERT INTO bookings.bookings_json (booking_id, data)
SELECT b.book_ref,
       jsonb_build_object(
           'booking_number', b.book_ref,
           -- Условный статус: в исходной таблице статуса брони нет.
           'status', 'confirmed',
           'passengers', COALESCE(p.passengers, '[]'::jsonb),
           'notes', ''
       )
FROM bookings.bookings AS b
LEFT JOIN passenger_lists AS p ON p.book_ref = b.book_ref
-- Повторный запуск сохраняет уже созданные документы и их notes.
ON CONFLICT (booking_id) DO NOTHING;

COMMIT;

SELECT (SELECT count(*) FROM bookings.bookings) AS source_bookings,
       count(*) AS json_bookings
FROM bookings.bookings_json;

SELECT booking_id, jsonb_pretty(data) AS booking_json
FROM bookings.bookings_json
ORDER BY booking_id
LIMIT 2;
```

Приложение Б

02_update_notes.sql

```
-- Этап 2: добавить комментарий для рейсов в/из аэропортов,
-- отличающихся от Москвы СТРОГО больше чем на 6 часов.
BEGIN;

WITH comment_text AS (
    SELECT 'Бронирование содержит рейс в/из аэропорта с разницей во времени относительно Москвы
    более 6 часов.'::text AS note
), affected_bookings AS (
    SELECT DISTINCT t.book_ref
    FROM bookings.tickets AS t
    JOIN bookings.ticket_flights AS tf ON tf.ticket_no = t.ticket_no
    JOIN bookings.flights AS f ON f.flight_id = tf.flight_id
    JOIN bookings.airports_data AS dep ON dep.airport_code = f.departure_airport
    JOIN bookings.airports_data AS arr ON arr.airport_code = f.arrival_airport
    WHERE
        -- Время одного и того же момента в двух поясах, на дату вылета.
        abs(extract(epoch FROM (
            (f.scheduled_departure AT TIME ZONE dep.timezone) -
            (f.scheduled_departure AT TIME ZONE 'Europe/Moscow')
        )) / 3600) > 6
        OR
        -- Для прилёта используем дату и время прилёта.
        abs(extract(epoch FROM (
            (f.scheduled_arrival AT TIME ZONE arr.timezone) -
            (f.scheduled_arrival AT TIME ZONE 'Europe/Moscow')
        )) / 3600) > 6
)
UPDATE bookings.bookings_json AS bj
SET data = jsonb_set(
    bj.data,
    '{notes}',
    to_jsonb(concat_ws(E'\n', NULLIF(bj.data ->> 'notes', ''), c.note)),
    true
)
FROM affected_bookings AS ab
CROSS JOIN comment_text AS c
WHERE bj.booking_id = ab.book_ref
  -- Не добавляем ту же фразу ещё раз при повторном запуске.
  AND position(c.note IN COALESCE(bj.data ->> 'notes', '')) = 0;

COMMIT;

SELECT count(*) AS bookings_with_comment
FROM bookings.bookings_json
WHERE position(
    'Бронирование содержит рейс в/из аэропорта с разницей во времени относительно Москвы более 6
    часов.'
    IN COALESCE(data ->> 'notes', '')
) > 0;

SELECT booking_id, data ->> 'notes' AS notes
FROM bookings.bookings_json
WHERE COALESCE(data ->> 'notes', '') <> ''
ORDER BY booking_id
LIMIT 5;
```